



Sistemas Embebidos I

Otoño 2026

**Práctica #3: Bare-Metal/O&Timing Timing&Sys
Tick; timestamping; pin-toggle/tracing**

María Fernanda Conde Calatayud

Andrea Ceja Lara

26 / 08 / 2026

Índice

<i>Práctica #3: Bare-Metall/O&Timing Timing&SysTick; timestamping; pin-toggletesting</i>	2
Introducción	2
Descripción de la práctica	2
Objetivos	2
Marco teórico	2
Desarrollo	3
Materiales	3
Esquemático	4
Desarrollo	4
Resultados	5
Evidencias	5
Conclusiones	5

Práctica #3: Bare-Metall/O&Timing

Timing&SysTick; timestamping; pin-toggletiming

Introducción

Descripción de la práctica

La práctica consiste en implementar un temporizador de sistema utilizando interrupciones (IRQ) para generar una señal mediante *toggling* y observar su comportamiento con un osciloscopio. También se busca medir el jitter de la señal para comparar el valor esperado con el valor real obtenido.

Objetivos

- Implementar un temporizador utilizando interrupciones IRQ.
- Generar una señal periódica mediante toggling de un GPIO.
- Medir el periodo de la señal utilizando un osciloscopio.
- Calcular y analizar el jitter de la señal.
- Comparar los valores programados en el código con los valores medidos físicamente.

Marco teórico

Timers y contadores

Los timers son recursos del microcontrolador que permiten medir el paso del tiempo sin tener que detener completamente la ejecución del programa. Funcionan utilizando un contador que aumenta de acuerdo con el reloj del sistema y permite generar eventos después de un determinado periodo (1).

Los contadores permiten llevar un registro de unidades de tiempo o de eventos. En conjunto con los timers, pueden utilizarse para realizar acciones periódicas, como cambiar el estado de un GPIO cada cierto tiempo. En modo μs (1 tick = 1 μs), eso tardaría en dar la vuelta (overflow) aproximadamente 584,942 años. Para efectos prácticos, el contador de 64 bits nunca se desborda durante la vida útil de cualquier proyecto (2).

En esta práctica, el temporizador se utiliza para generar cambios periódicos en una salida y obtener una señal que posteriormente será medida con el osciloscopio.

Alarmas

Las alarmas permiten programar una acción para que ocurra después de cierto tiempo. Cuando el temporizador alcanza el valor establecido, se genera un evento que puede activar una función o una interrupción.

Esto permite realizar tareas periódicas sin utilizar retardos bloqueantes como `sleep()`, haciendo que el microcontrolador pueda continuar realizando otras operaciones mientras espera el momento programado (3).

Interrupciones (IRQ)

Las interrupciones, también conocidas como IRQ (*Interrupt Requests*), permiten que el microcontrolador responda inmediatamente a determinados eventos. Cuando ocurre una interrupción, el procesador pausa temporalmente la ejecución normal del programa y ejecuta una función específica llamada rutina de servicio de interrupción (ISR) (3).

En esta práctica, la interrupción se genera cuando el temporizador alcanza el tiempo programado. La ISR se encarga de cambiar el estado del GPIO (*toggling*), generando así una señal periódica.

El uso de interrupciones permite implementar temporizadores de una manera más eficiente, ya que no es necesario mantener al procesador detenido esperando a que transcurra el tiempo.

Jitter

El jitter es la variación que existe en el tiempo entre eventos que deberían ocurrir de manera periódica, nunca es 0 (2). Por ejemplo, si se programa una señal para cambiar de estado cada 250 ms, idealmente cada cambio debería ocurrir exactamente en ese intervalo. Sin embargo, factores como las interrupciones, la ejecución del programa y el funcionamiento del microcontrolador pueden generar pequeñas variaciones.

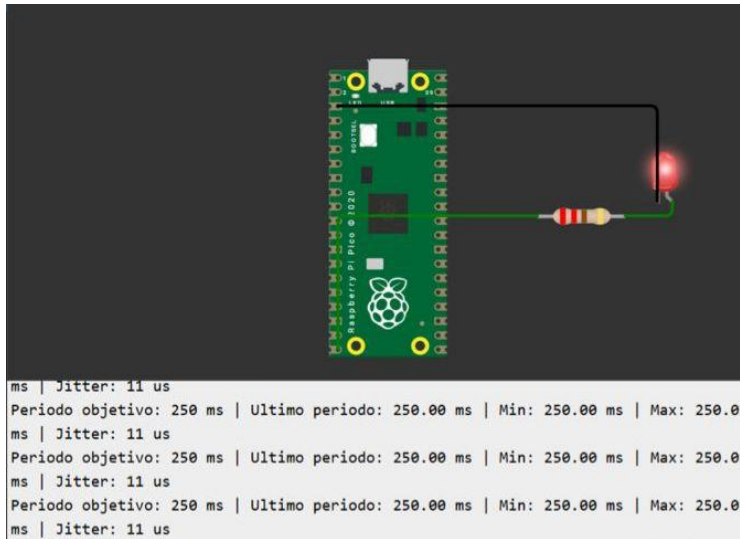
En esta práctica, el jitter se puede observar utilizando un osciloscopio, comparando los diferentes periodos de la señal. Mientras menor sea la variación entre ellos, menor será el jitter y más estable será el temporizador.

Desarrollo

Materiales

- Raspberry Pi Pico
- Diodo LED
- Protoboard
- Jumpers
- Resistencia 220 ohms
- Simulador wowki

Esquemático



Desarrollo

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/timer.h"
#include "hardware/irq.h"

#define ALARM_NUM 0 //define el numero de alarma
#define ALARM_IRQ TIMER_IRQ_0 //define la interrupcion del timer
#define INTERVAL_US 500000 //define el intervalo de tiempo en microsegundos
#define TRACE_PIN 14 //define el gpio para la traza del osciloscopio
#define LED_PIN 15 //define el pin del led externo
volatile uint32_t next_alarm; //guarda el siguiente tiempo de alarma

void on_alarm_irq() {
    hw_clear_bits(&timer_hw->intr, 1u << ALARM_NUM); //limpia la interrupcion de la alarma
    gpio_xor_mask(1u << LED_PIN); //cambia el estado del led externo
    gpio_xor_mask(1u << TRACE_PIN); //cambia el estado del gpio14
    next_alarm += INTERVAL_US; //calcula el siguiente tiempo de alarma
    timer_hw->alarm[ALARM_NUM] = next_alarm; //programa la siguiente alarma
}

int main() {
    stdio_init_all(); //inicia la comunicacion serial
    gpio_init(LED_PIN); //inicia el pin del led externo
    gpio_set_dir(LED_PIN, GPIO_OUT); //configura el led como salida
    gpio_put(LED_PIN, 0); //inicia el led apagado
    gpio_init(TRACE_PIN); //inicia el pin de la traza
    gpio_set_dir(TRACE_PIN, GPIO_OUT); //configura el gpio14 como salida
    gpio_put(TRACE_PIN, 0); //inicia la traza en bajo
    irq_set_exclusive_handler(ALARM_IRQ, on_alarm_irq); //asigna la funcion a la interrupcion
    hw_set_bits(&timer_hw->inte, 1u << ALARM_NUM); //habilita la alarma del timer
    irq_set_enabled(ALARM_IRQ, true); //habilita la interrupcion en el nucleo

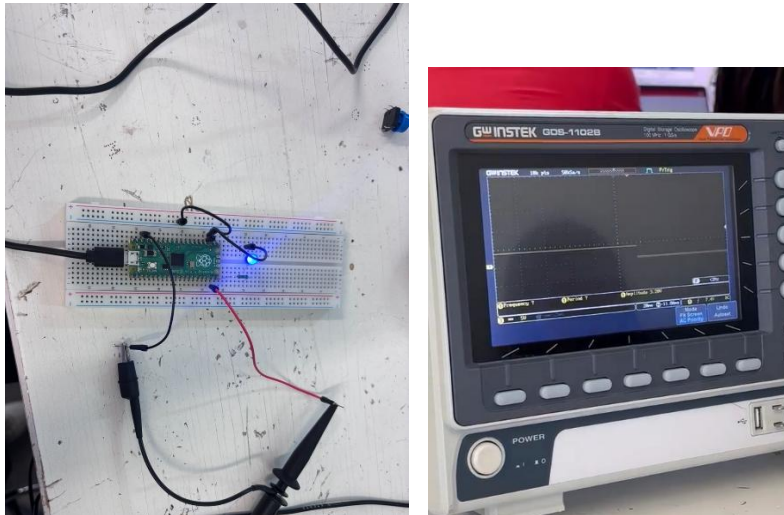
    next_alarm = timer_hw->timerawl + INTERVAL_US; //calcula el primer tiempo de alarma
    timer_hw->alarm[ALARM_NUM] = next_alarm; //programa la primera alarma

    while (true) {
        tight_loop_contents(); //mantiene el programa principal activo
    }
    return 0;
}
```

Resultados

Parámetro	Valor esperado (del código)	Valor medido (osciloscopio)
Periodo	(el que se programa, i.e. 250 ms)	Min= 250 ms Max=250.01 ms
Jitter (Max – Min, o Std)	≈ 0 (ideal)	11 us

Evidencias



Conclusiones

En esta práctica pude comprobar que el uso de interrupciones (IRQ) junto con los timers y las alarmas permite generar una señal periódica bastante estable. Al medirla con el osciloscopio, el periodo obtenido fue muy cercano al programado, mostrando que el jitter fue mínimo.

También entendí que programar cada alarma tomando como referencia el intervalo anterior ayuda a evitar que se acumulen pequeños retrasos. En general, la práctica me ayudó a comprender mejor cómo funcionan los timers, alarmas e interrupciones, y por qué son importantes cuando se necesita precisión en el tiempo.

Bibliografías

- [1] S. B. Reddy, «Timers and Counters - Automation Community», *Automation Community*, 22 de junio de 2021. <https://learn.automationcommunity.com/timers-and-counters/>
- [2] «Bare-Metall/O&Timing Timing&SysTick; Timestamping; Pin-ToggleTracing», 26 de agosto de 2026.
- [3] OpenAI. (2026). *ChatGPT* [Modelo de lenguaje]. <https://chatgpt.com>